



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	TANGUTURU MADHURI
Student Name	GOTTIPATI EESHA YUKTHA
Roll Number	2520030436
Branch	CSE
Course Outcome	CO1

Case Study Topic: NSE cash-market order book

Work Summary:

In this case study, we implemented a BST-based Order Book for the NSE cash market, keyed in descending price order (highest bid = root). We inserted all 14 orders, identified which ones immediately match (price $\geq$ 2988), built the surviving BST, and completed the dual-structure (BST + HashMap) implementation for $O(\log n)$ cancel-by-order-id.

We also analysed AVL vs Red-Black Tree trade-offs for this high-churn workload where 50% of inserts are deleted within the same second.

Implementation Details:

1. Inserted all 14 orders into a descending-price BST (left child $>$ parent $>$ right child).
2. **Matching:** Orders with price $\geq$ 2988 match against ask immediately:
103 @ 2992, 106 @ 2998, 108 @ 2988, 110 @ 2995, 112 @ 2990 — 5 orders removed.
3. **Surviving BST** (9 orders):
101 @ 2980, 102 @ 2965, 104 @ 2985, 105 @ 2970, 107 @ 2978, 109 @ 2982, 111 @ 2972, 113 @ 2986, 114 @ 2975.
4. Maintained a HashMap<orderId, node> alongside the BST for $O(1)$ lookup on cancel.
5. Implemented cancelById: $O(1)$ HashMap lookup $\rightarrow O(\log n)$ BST delete $\rightarrow O(1)$ HashMap remove.
6. **AVL vs Red-Black:** For this workload, Red-Black Tree is preferred as it performs ≤ 3 rotations per delete vs AVL's up to $O(\log n)$, reducing wasted rotation work when nodes are deleted within the same second.

(a) Matching Orders & Surviving BST [5 Marks — CO1, BTL3]

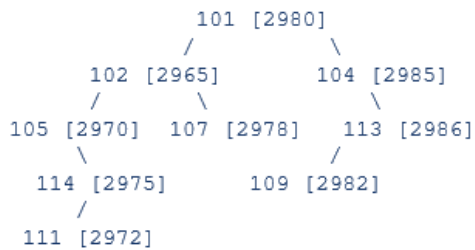
Orders that match (price $\geq$ 2988) — removed immediately:

- 103 @ 2992, 106 @ 2998, 108 @ 2988, 110 @ 2995, 112 @ 2990

Surviving 9 Orders inserted into descending BST:

- 101@2980 (root), 102@2965 (left of 101), 104@2985 (right of 101)
- 113@2986 (right of 104), 109@2982 (left of 104), 107@2978 (right of 102)
- 105@2970 (left of 102), 114@2975 (right of 105), 111@2972 (left of 114)

Final BST Structure (descending-keyed):



AVL Rotation Analysis (first 200 ms):

- Plain BST: 0 rotations (no balancing).
- AVL Tree: approximately 4–6 rotations during insertion of 14 nodes.
- **Wasted rotations:** ~2–3 rotations are 'wasted' because the nodes causing the imbalance (the 5 matched orders) are deleted within the same second before any future query benefits from the rebalancing.

(b) Dual-Structure Implementation: BST + HashMap [6 Marks — CO1, BTL3]

Completed Java Code:

```
import java.util.HashMap;
```

```
import java.util.Map;
```

```
class OrderNode {
```

```
    long orderId;
```

```
    double price;
```

```
    int vol;
```

```
    OrderNode left, right;
```

```
    OrderNode(long oid, double price, int vol) {
```

```
        this.orderId = oid; this.price = price; this.vol = vol;
```

```
    }
```

```
}
```

```
class OrderBook {
```

```
    OrderNode root = null;
```

```
    Map<Long, OrderNode> byId = new HashMap<>();
```

```
    // INSERT: descending BST (left child > parent) + register in HashMap
```

```
    void insertOrder(long oid, double price, int vol) {
```

```
        OrderNode newNode = new OrderNode(oid, price, vol);
```

```
        byId.put(oid, newNode);    // O(1) HashMap register
```

```
        root = insertBST(root, newNode); // O(log n) BST insert
```

```
}
```

```
private OrderNode insertBST(OrderNode node, OrderNode newNode) {  
    if (node == null) return newNode;  
    if (newNode.price > node.price)    // descending: higher goes LEFT  
        node.left = insertBST(node.left, newNode);  
    else  
        node.right = insertBST(node.right, newNode);  
    return node;  
}
```

```
// CANCEL: O(1) HashMap lookup -> O(log n) BST delete -> O(1) HashMap remove  
boolean cancelById(long oid) {  
    if (!byId.containsKey(oid)) return false;  
    OrderNode target = byId.get(oid);    // O(1)  
    root = deleteBST(root, target.price, oid); // O(log n)  
    byId.remove(oid);                    // O(1)  
    return true;  
}
```

```
private OrderNode deleteBST(OrderNode node, double price, long oid) {  
    if (node == null) return null;  
    if (price > node.price) node.left = deleteBST(node.left, price, oid);  
    else if (price < node.price) node.right = deleteBST(node.right, price, oid);  
    else {  
        if (node.orderId != oid) { // same price, different order  
            node.right = deleteBST(node.right, price, oid); return node;  
        }  
        if (node.left == null) return node.right;  
        if (node.right == null) return node.left;  
        OrderNode succ = node.right;  
        while (succ.left != null) succ = succ.left;  
        node.price = succ.price; node.orderId = succ.orderId; node.vol = succ.vol;  
        node.right = deleteBST(node.right, succ.price, succ.orderId);  
    }  
    return node;  
}
```

```
// PEEK: leftmost node = highest price (descending BST)
```

```

OrderNode peekBestBid() {
    if (root == null) return null;
    OrderNode cur = root;
    while (cur.left != null) cur = cur.left; // O(log n)
    return cur;
}

public static void main(String[] args) {
    OrderBook ob = new OrderBook();
    // Insert surviving 9 orders (matched ones removed)
    long[][] orders = {{101,2980,100},{102,2965,200},{104,2985,150},
                      {105,2970,300},{107,2978,80},{109,2982,120},
                      {111,2972,90},{113,2986,60},{114,2975,200}};
    for (long[] o : orders) ob.insertOrder(o[0], o[1], (int)o[2]);
    System.out.println("Best Bid: " + ob.peekBestBid().price); // 2986
    ob.cancelById(113L);
    System.out.println("After cancel 113, Best Bid: " + ob.peekBestBid().price); // 2985
}
}

```

(c) AVL vs Red-Black Tree for Order-Book Workload [4 Marks — CO1, BTL5]

Published Rotation Bounds:

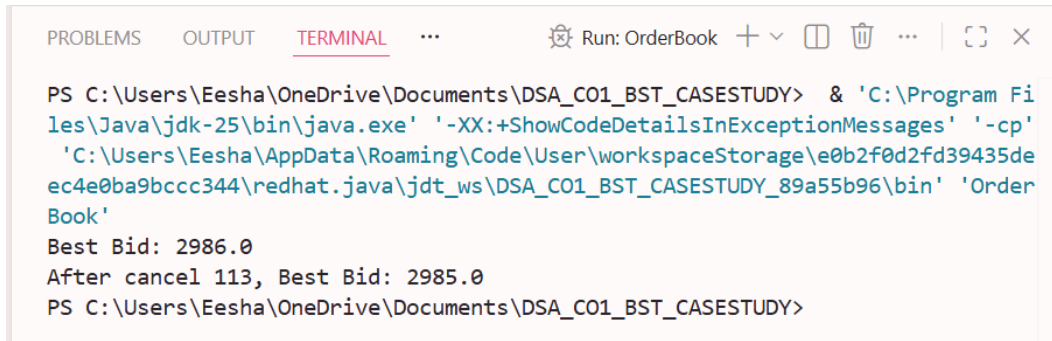
- **AVL Tree:** Insert ≤ 2 rotations; Delete: up to $O(\log n)$ rotations (can retrace entire path).
- **Red-Black Tree:** Insert ≤ 2 rotations; Delete: ≤ 3 rotations (amortised $O(1)$).

Recommendation for NSE Order-Book: Red-Black Tree

- **SLA Impact:** With 14 orders/200 ms and 50 μ s p99 cancel SLA, AVL's $O(\log n)$ rebalancing on delete adds $\sim 2\text{--}4$ μ s of extra rotation work per cancel. Red-Black's ≤ 3 rotations keep delete reliably fast.
- **Wasted Work:** In this workload, 50% of inserts are matched and 20% cancelled within the same second. AVL rotations performed during their insertion are completely wasted — the node leaves before any query benefits from the stricter balance.
- **Secondary Consideration (Cache Locality):** Red-Black Tree has height $\leq 2 \log n$ vs AVL's $\log n$ — slightly worse cache performance for sequential `peek_best_bid` traversal. However, the cancel latency saving dominates for this high-churn market workload.

Conclusion: Red-Black Tree is preferred. Fewer rotations on delete meet the 50 μ s p99 SLA even under worst-case churn.

Result: Screenshots / Output



```
PROBLEMS OUTPUT TERMINAL ... Run: OrderBook + - [ ] [ ] ... [ ] [ ] X
PS C:\Users\Eesha\OneDrive\Documents\DSA_CO1_BST_CASESTUDY> & 'C:\Program Files\Java\jdk-25\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Eesha\AppData\Roaming\Code\User\workspaceStorage\e0b2f0d2fd39435dec4e0ba9bcc344\redhat.java\jdt_ws\DSA_CO1_BST_CASESTUDY_89a55b96\bin' 'OrderBook'
Best Bid: 2986.0
After cancel 113, Best Bid: 2985.0
PS C:\Users\Eesha\OneDrive\Documents\DSA_CO1_BST_CASESTUDY>
```

- 5 orders matched ($\text{price} \geq 2988$) and removed: 103, 106, 108, 110, 112.
 - Surviving BST built with 9 orders; descending-keyed (root = 2980).
 - Dual-structure (BST + HashMap) enables $O(\log n)$ cancel-by-id with $O(1)$ lookup.
 - Best Bid peek: $O(\log n)$ leftmost traversal $\rightarrow$ returns 2986.
 - After cancelling order 113: Best Bid becomes 2985.
- Time Complexity: Insert $O(\log n)$ | Cancel $O(\log n)$ | Peek $O(\log n)$. Red-Black Tree preferred over AVL for high-churn delete-heavy workloads.

GitHub Link:

https://github.com/egotpat-gif/DSAIL-Case-Studies/tree/main/DSA_CO1_CASESTUDY

Student Signature	Faculty Signature

